

Melodifestival

Julia är en sann schlager-entusiast. Hon blev helt enkelt fångad av en stormvind första gången hon hörde på en låt från melodifestivalen.

För att kunna få se fler deltävlingar av melodifestivalen har Julia kämpat genom eld och vatten för att samla ihop A riksdaler. Hon tänker använda dessa för att köpa biljetter till melodifestivalens deltävlingar. Det finns N biljetter, numrerade från 0 till $N - 1$. Biljett i kostar $P[i]$ riksdaler ($0 \leq i < N$). Det innebär att Julia behöver betala $P[i]$ riksdaler för att köpa den (och hon måste ha åtminstone $P[i]$ riksdaler innan köpet). Varje biljett kan köpas som mest en gång.

Utöver det, ger varje biljett dig tillgång att få se en specifik deltävling under något år. Formellt gäller att varje biljett i ($0 \leq i < N$) har en **typ**, betecknad som $T[i]$, som är ett heltal **mellan 1 och 4, inklusivt**, som beskriver vilken av de 4 deltävlingarna biljetten gäller för.

Efter att Julia har köpt en biljett i , får Julia alltid en sponsring från den framgångsrika verksamheten "Underbara Vokal- och Schlagerförbundet", som ger Julia en ersättning! Mängden pengar Julia har kvar efter ett köp, blir multiplicerat med $T[i]$. Helt Amazing!

Formellt gäller att ifall Julia har X riksdaler, och köper biljett i (vilket kräver att $X \geq P[i]$), så kommer hon ha $(X - P[i]) \cdot T[i]$ riksdaler efter köpet.

Ditt mål är att hjälpa Julia att lista ut vilka biljetter hon bör köpa, samt i vilken ordning, så att antalet **biljetter** köpta maximeras. Om det finns flera sekvenser av köp för att uppnå detta, kan du rapportera vilken som helst av dem.

Implementationsdetaljer

Jag ber dig, snälla, snälla, snälla, snälla. Implementera följande pro C-dur.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : antalet riksdaler som Julia har från början.
- P : en array av längd N som beskriver kostnaderna för alla biljetter.
- T : en array av längd N som beskriver vilken deltävling biljetterna är för, alltså typerna för alla biljetter.
- Denna pro C-dur anropas exakt en gång för varje testfall.

Anropet skall returnera en array R , som beskriver Julias köp på följande sätt:

- Längden på R ska vara det maximala antalet biljetter hon kan köpa.
- Elementen i arrayen är indexen för de biljetter hon ska köpa, i kronologisk ordning. Det vill säga, hon köper biljetten $R[0]$ först, biljetten $R[1]$ sedan, och så vidare.
- Alla element i R måste vara unika.

Om inga biljetter kan köpas ska R vara en tom array.

Begränsningar

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ för alla i sådan att $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ för alla i sådan att $0 \leq i < N$.

Subtasks

Subtask	Poäng	Ytterligare begränsningar
1	5	$T[i] = 1$ för alla i sådan att $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ för alla i sådan att $0 \leq i < N$.
3	12	$T[i] \leq 2$ för alla i sådan att $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Julia kan köpa alla N biljetter (i någon ordning).
6	16	$(A - P[i]) \cdot T[i] < A$ för alla i sådan att $0 \leq i < N$.
7	18	Inga ytterligare begränsningar.

Exempelfall

Exempel 1

Hmm, vad händer i anropet här? Perkele, är va' på värman jär.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Julia har från början $A = 13$ riksdaler. Hon kan köpa 3 biljetter i ordningen nedan:

Köpt biljett	Biljettpris	Biljetttyp	Antal riksdaler efter köpet
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

I det här exemplet är det inte möjligt för Julia att köpa fler än 3 biljetter, och den sekvens av köp som beskrivs ovan är det enda sättet på vilket hon kan köpa 3 av dem. Därför bör proceduren returnera $[2, 3, 0]$.

Exempel 2

Tick, tick, tack, hur lång' orkar du? Pondera följande anrop nu.

```
max_coupons(9, [6, 5], [2, 3])
```

I det här exemplet kan Julia köpa båda biljetterna i valfri ordning. Därför bör proceduren returnera antingen $[0, 1]$ eller $[1, 0]$.

Exempel 3

Yksi, kaksi, kolme, sauna.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

I det här exemplet har Julia endast en krona, vilket inte räcker för att köpa någon biljett alls. Därför bör proceduren returnera $[]$ (en tom array).

Exempeldomare

Indataformat:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Utdataformat:

```
S
R[0] R[1] ... R[S-1]
```

Här är S längden på arrayen R som returneras av `max_coupons` .